

TERRAFORM_01_TASK

What is Terraform?

Terraform is an **Infrastructure as Code (IaC)** tool used to **create, update, and manage infrastructure automatically** using configuration files instead of manual clicks.

You write infrastructure in **code**, Terraform builds it for you.

Uses of Terraform (What we use it for)

1 Provision Cloud Infrastructure

Create resources like:

- EC2 / VM
- VPC / Networks
- Load Balancers
- RDS / Databases
- S3 / Storage
- Kubernetes clusters (EKS, AKS, GKE)

 Works with **AWS, Azure, GCP**, and many more.

2 Infrastructure Automation

Instead of:

- Clicking in AWS Console ❌
- Manual mistakes ❌

You do:

terraform apply

✓ Everything is created automatically.

3 Multi-Cloud Infrastructure

One tool for:

- AWS
 - Azure
 - GCP
 - On-prem
 - SaaS tools (GitHub, Datadog, Cloudflare)
-

CI/CD Integration

Terraform is widely used in:

- Jenkins
- GitHub Actions
- GitLab CI

Example:

- Pipeline deploys infra before app deployment
-

5 Infrastructure Version Control

Terraform files are:

- Stored in Git
- Versioned
- Reviewed via PRs
- Rolled back if needed

Benefits of Terraform

1. Infrastructure as Code (IaC)

- ✓ Infra written as **code**
- ✓ Reusable
- ✓ Reviewable
- ✓ Repeatable

Same infra in **Dev / QA / Prod** with small changes.

◆ 2. Automation & Speed

- Manual setup: 🕒 hours/days
- Terraform: ⌚ minutes

terraform init

terraform plan

terraform apply

◆ 3. Consistency & No Human Errors

- Same config → same infra every time
- No “it worked in my account” issue

◆ 4. State Management

Terraform keeps a **state file**:

- Knows what is already created
- Updates only what changed
- Avoids duplicate resources

◆ 5. Idempotent (Safe to Run Multiple Times)

Running Terraform again:

- ❌ Won't recreate existing resources
- ✓ Changes only what is needed

◆ 6. Multi-Cloud & Provider Support

Terraform supports **1000+ providers**:

- Cloud
- Databases

- Monitoring tools
- DNS
- CI/CD tools

◆ 7. Cost & Resource Control

- Easily destroy unused infra:

terraform destroy

- Prevents cloud cost leakage
-

◆ 8. Works Well with DevOps Tools

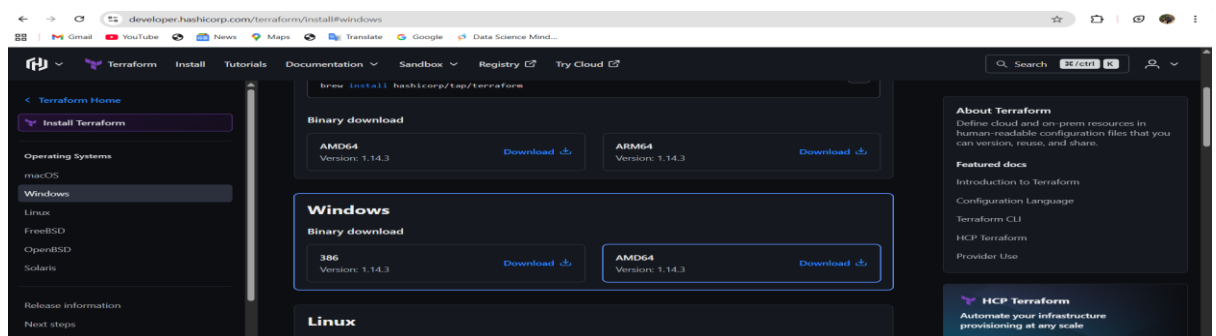
Terraform fits perfectly with:

- Docker
- Kubernetes
- Jenkins
- Ansible
- Git

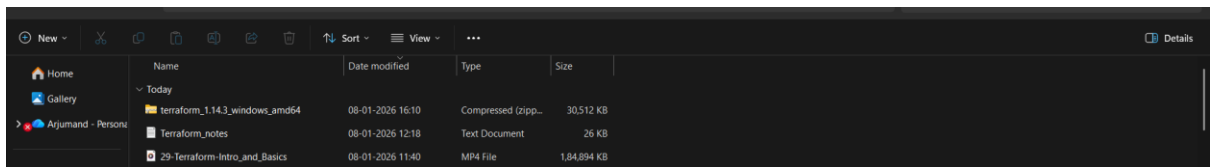
1. Install Terraform on your PC Execute all the templates shown in video.

Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider.

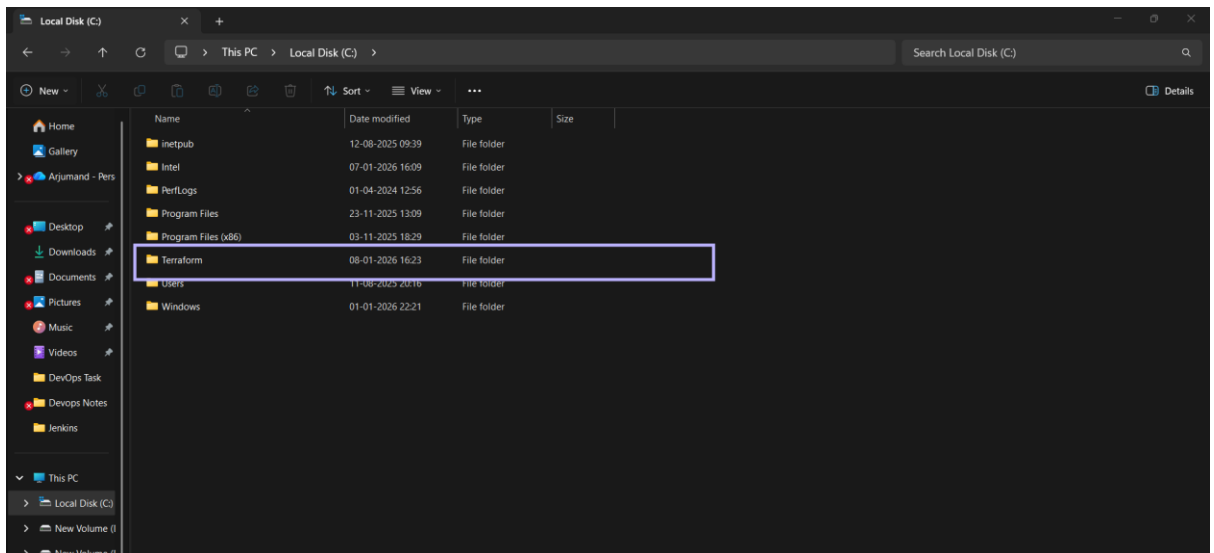
- Go to official terraform site and install terraform for desktop, I have selected 64



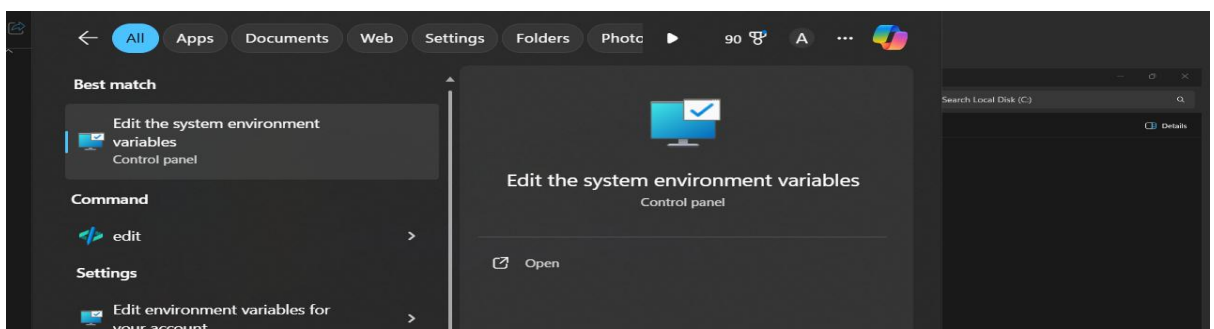
- After downloading terraform extract the file and open terraform and copy the path



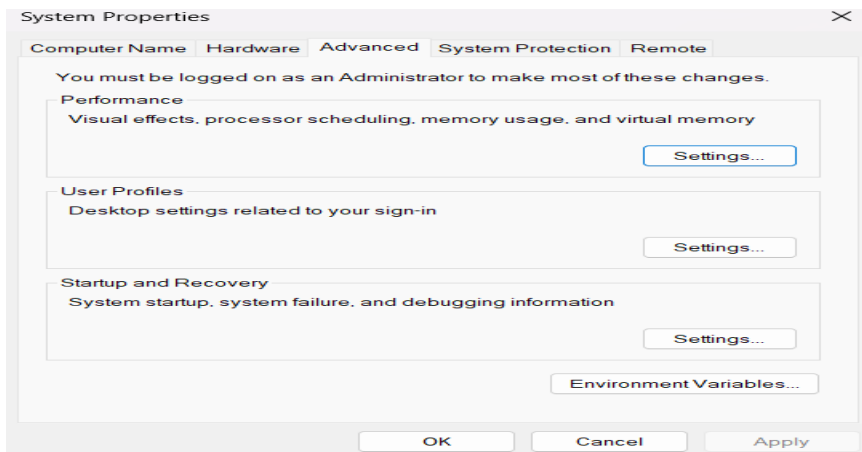
- In windows c create a folder and past that file in terraform folder



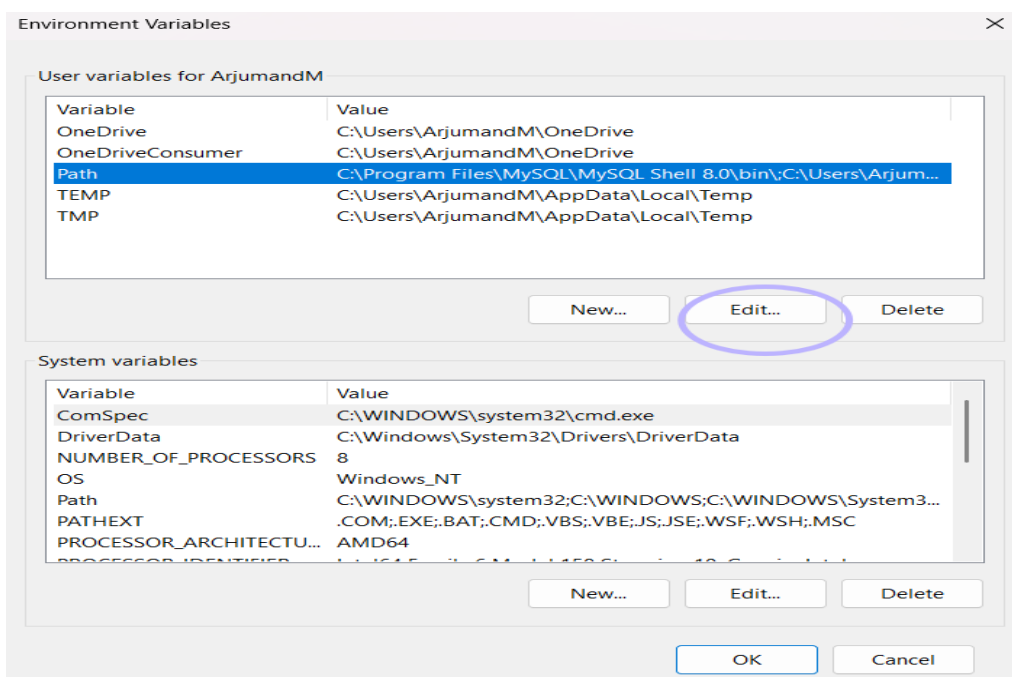
- Go to edit system variable



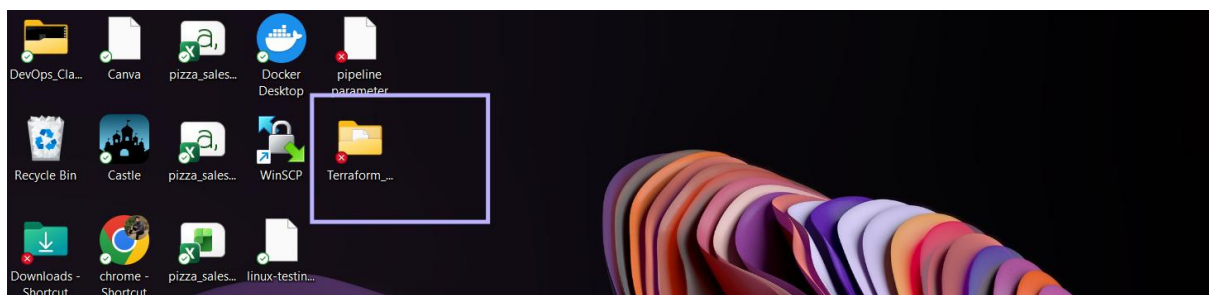
- Select environment variable



Select path and edit path and paste the path



Now create a folder at desktop 'terraform_basic'

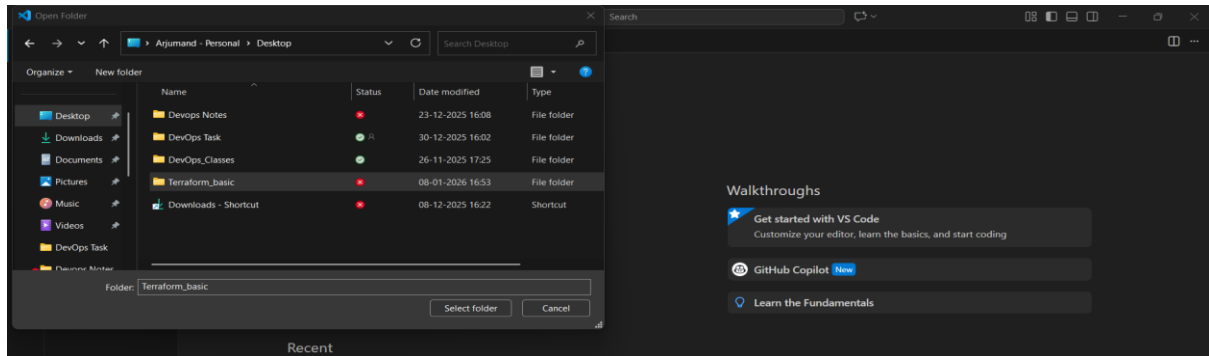


Check terraform is installed or not

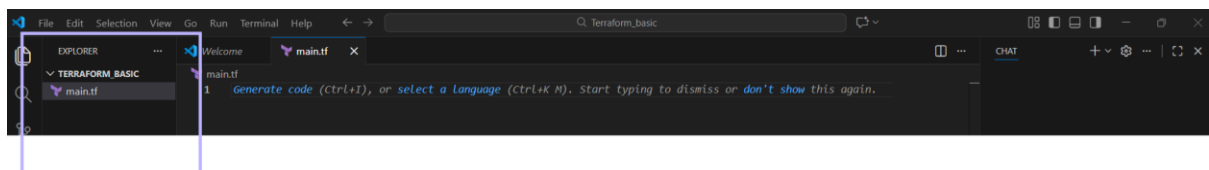
```
version      Show the current Terraform version
workspace    Workspace management

Global options (use these before the subcommand, if any):
-chdir=DIR   Switch to a different working directory before executing the
             given subcommand.
-help        Show this help output or the help for a specified subcommand.
-version     An alias for the "version" subcommand.
PS C:\Users\ArjumandM> terraform version
Terraform v1.14.3
on windows_amd64
PS C:\Users\ArjumandM> |
```

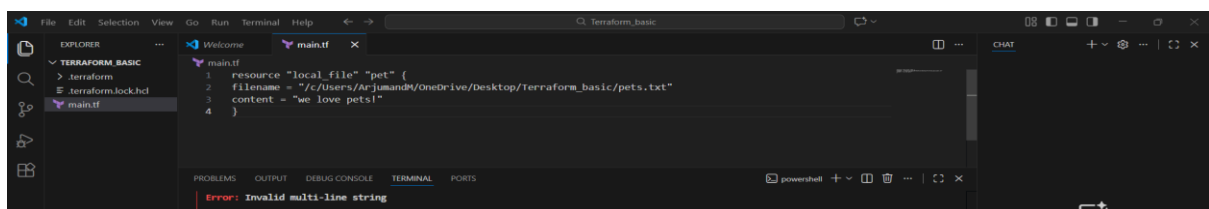
Go to visual studio and select folder select desktop location and then select terraform_basic and create a new file under that



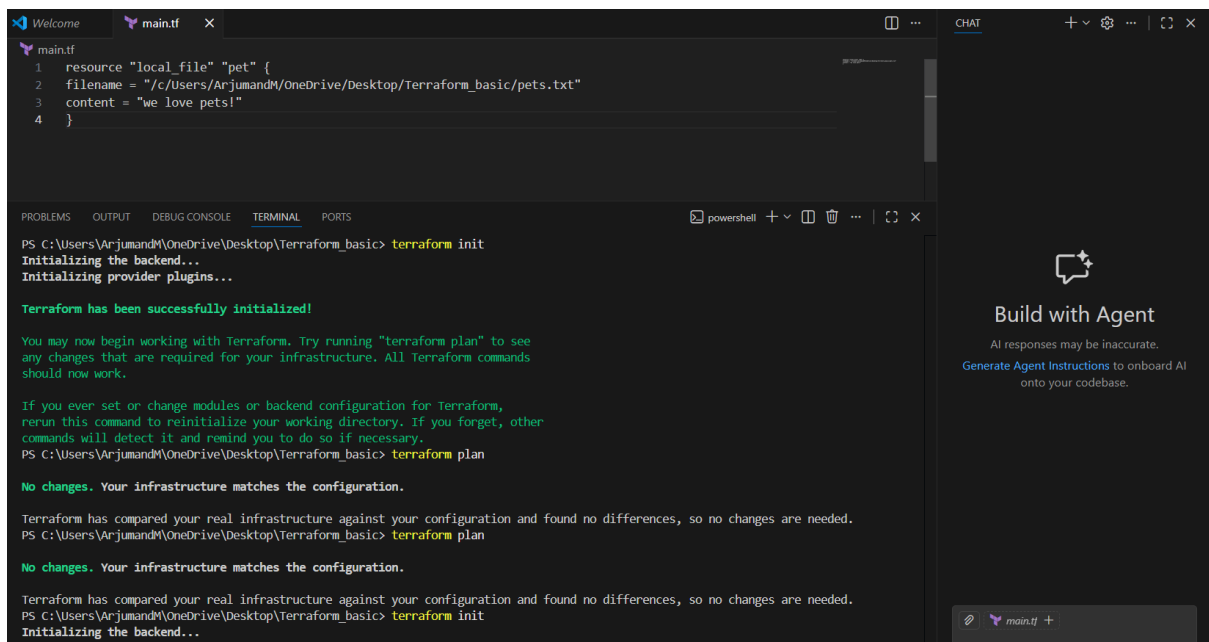
Created file 'main.tf'



Provide a small script



Now execute terraform init, terraform plan, terraform apply, and terraform destroy



The screenshot shows a VS Code editor with a file named `main.tf` containing Terraform configuration for a local file resource. The terminal window shows the execution of `terraform init` and `terraform plan`. The output of `terraform init` shows successful initialization. The output of `terraform plan` shows that no changes are needed for the infrastructure.

```
1 resource "local_file" "pet" {
2   filename = "/c/Users/ArjumandM/OneDrive/Desktop/Terraform_basic/pets.txt"
3   content = "we love pets!"
4 }
```

```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...
Initializing provider plugins...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform plan

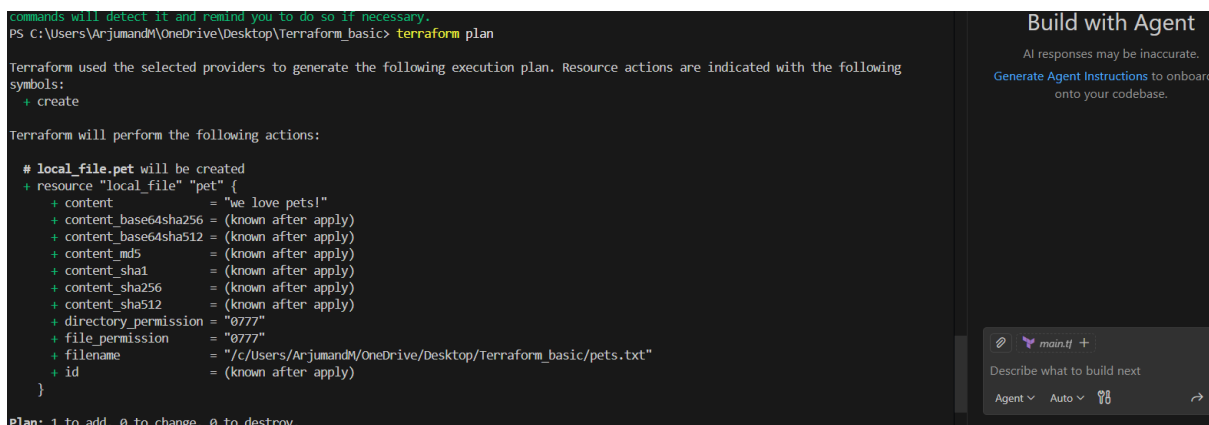
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform plan

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...
```

After making changes enter terraform plan



The screenshot shows the output of the `terraform plan` command. It displays the execution plan for the `local_file.pet` resource, indicating that it will be created. The plan shows the content of the file and the permissions. The output also includes a summary of the actions to be performed.

```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform plan

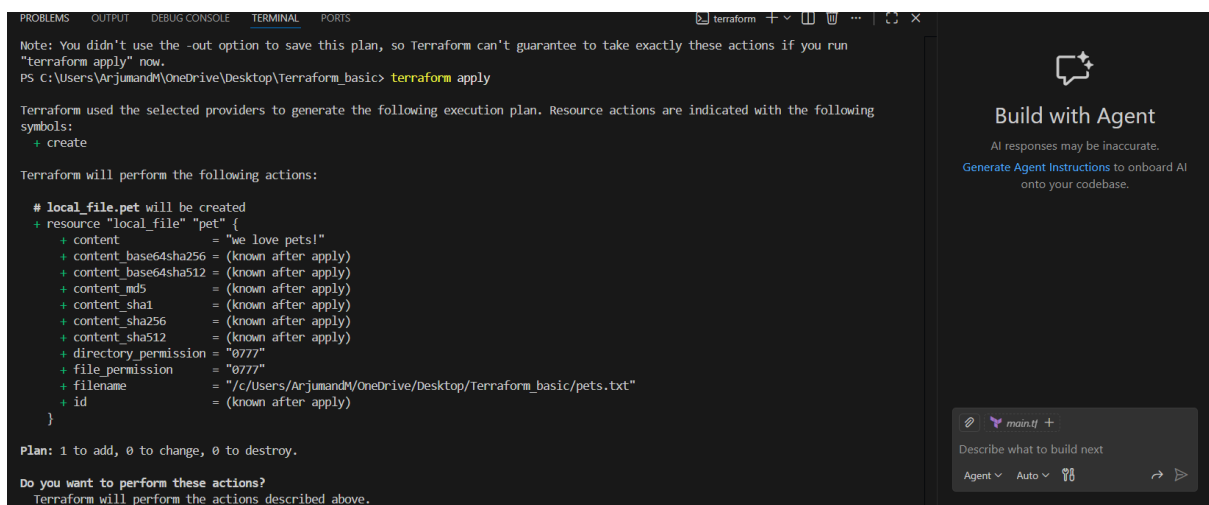
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# local_file.pet will be created
+ resource "local_file" "pet" {
  + content          = "we love pets!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "/c/Users/ArjumandM/OneDrive/Desktop/Terraform_basic/pets.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

After making changes enter terraform apply



The screenshot shows the execution of the `terraform apply` command. It displays the execution plan for the `local_file.pet` resource, indicating that it will be created. The plan shows the content of the file and the permissions. The output also includes a summary of the actions to be performed.

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run
"terraform apply" now.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# local_file.pet will be created
+ resource "local_file" "pet" {
  + content          = "we love pets!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "/c/Users/ArjumandM/OneDrive/Desktop/Terraform_basic/pets.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
```

Then enter terraform destroy.

```
Apply cancelled.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform destroy

No changes. No objects need to be destroyed.

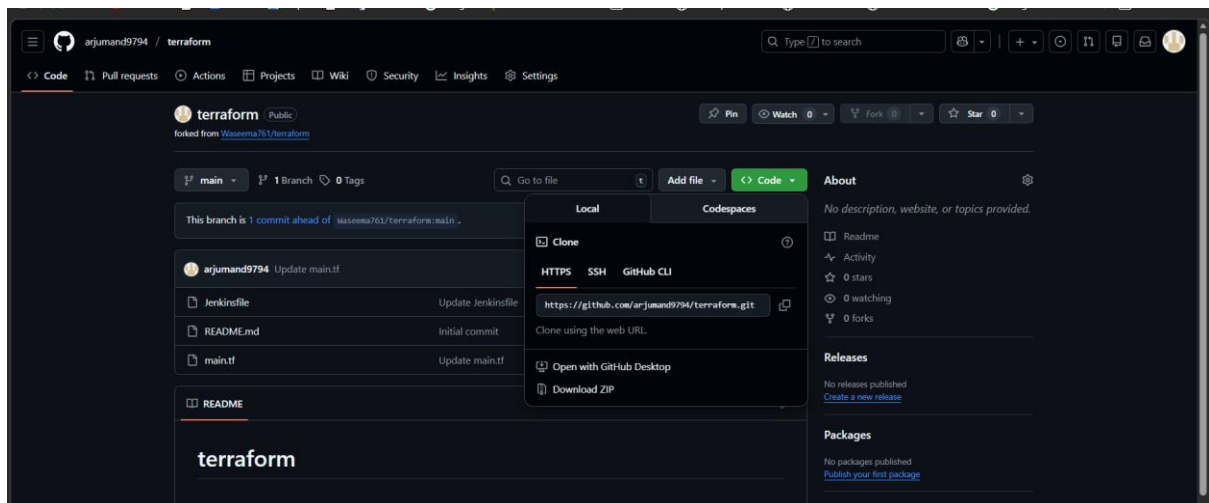
Either you have not created any objects yet or the existing objects were already deleted outside of Terraform.

Destroy complete! Resources: 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform destroy
```

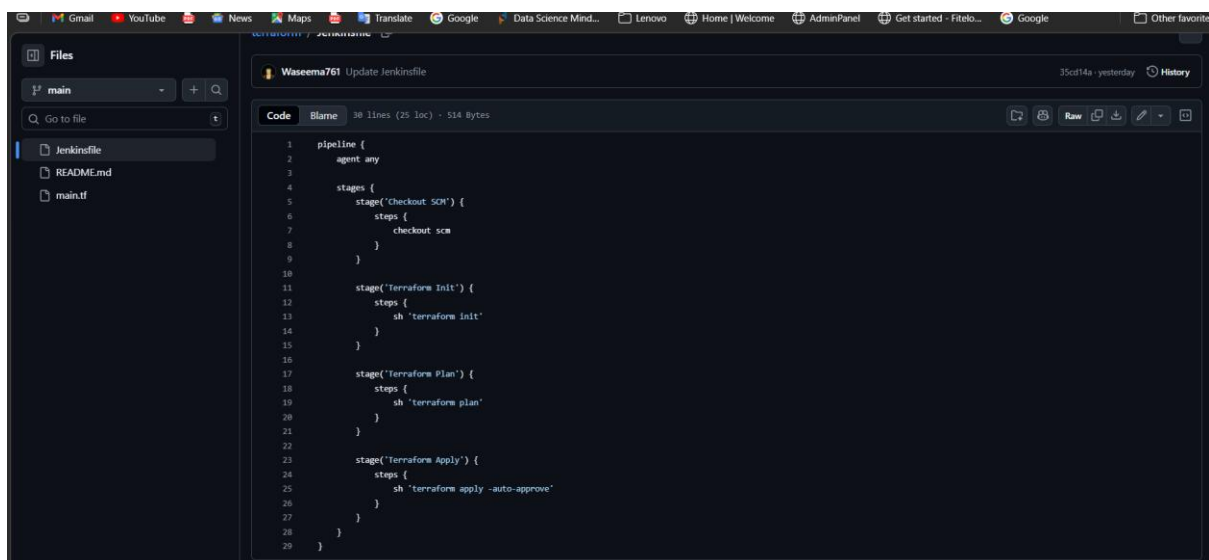
2. Integrate a sample Terraform template in Jenkins.

- First, we need to install terraform in our cli Jenkeins server
- Then add plugins in our Jenkins server

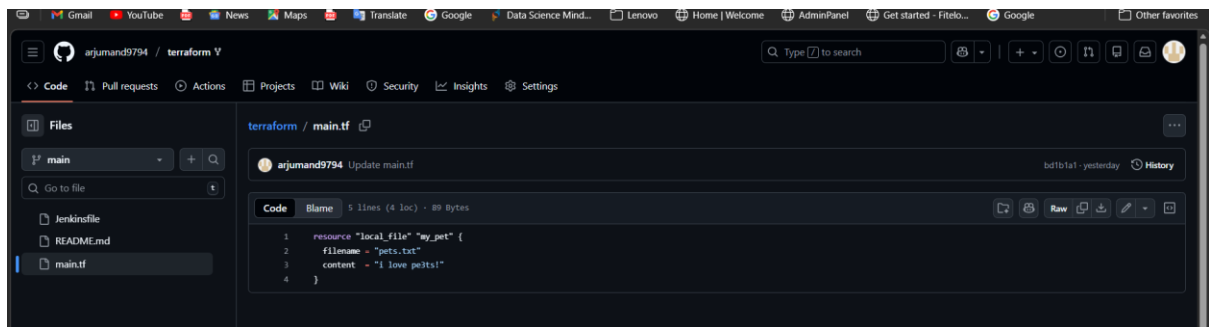
Created a repo and created two files in that repo



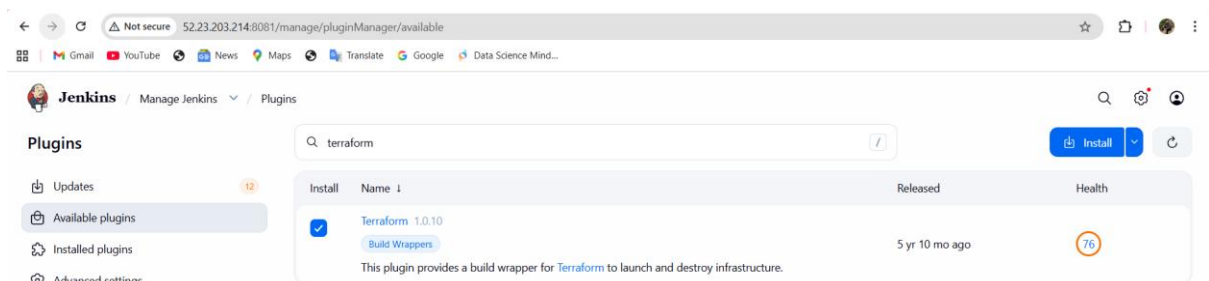
In Jenkinsfile I have created a script to execute



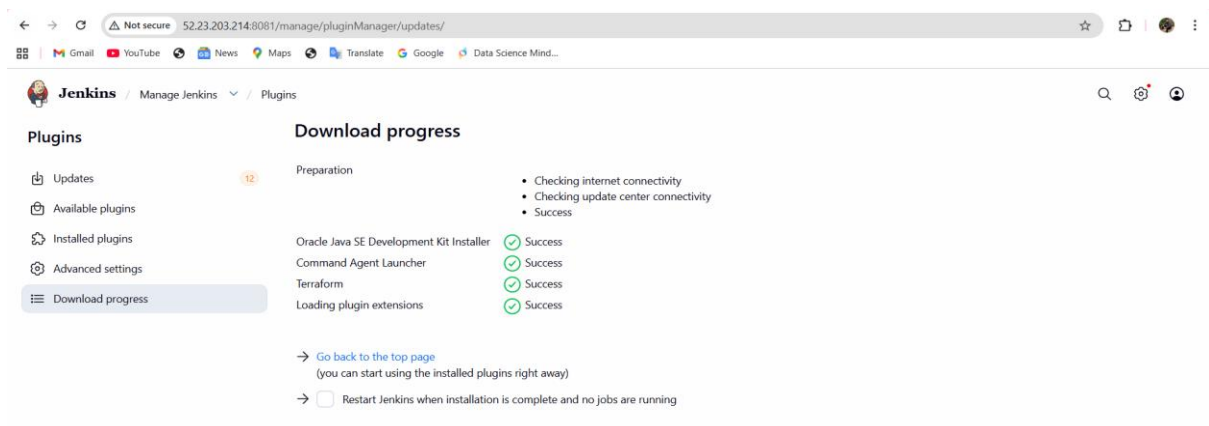
I have created one more file and added text in it



Created a job with pipeline



Job was executed successfully



These following details I have added

Name

Terraform

☒ Install automatically ?

Install from bintray.com

Version

Terraform 51203 darwin (amd64)

+ Add Installer

+ Add Terraform

Save

Apply

Select job name as terraform

/ All / New Item

Search

Settings

User

New Item

Enter an item name

Terraform_job

Select an item type

 Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

 Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

 Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 Folder

OK

Executed the job

 Jenkins

Terraform_job

#1

Search

Settings

User

Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

#1 (Jan 8, 2026, 2:32:11 PM)

Add description

Keep this build forever

Started by user Arjumand Mohiuddin

Started 15 sec ago

Took 3.9 sec

This run spent:

- 14 ms waiting;
- 3.9 sec build duration;
- 3.9 sec total from scheduled to completion.

git

Revision: bd1b1a125b7b53718e36e81354feda9120f367a

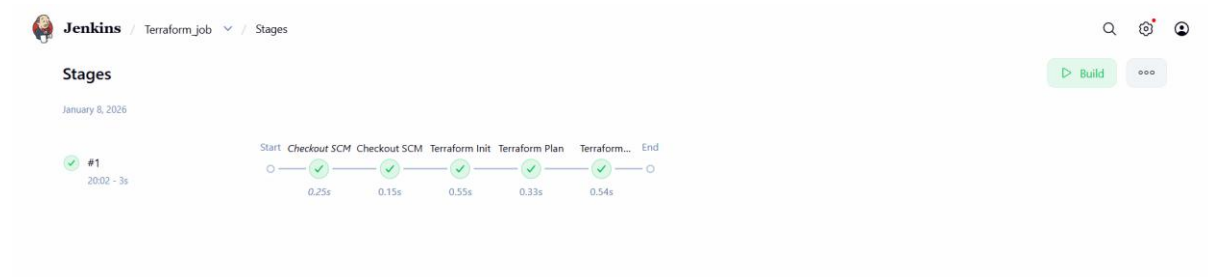
Repository: https://github.com/arjumand9794/terraform.git

- refs/remotes/origin/main

</>

No changes.

Stages of the task



Terraform apply stage as given below

← → ↺ Not secure 52.23.203.214:8081/job/Terraform_job/1/pipeline-overview/ ☆ 📄 👤

📧 Gmail 📺 YouTube 📰 News 🗺 Maps 🗨 Translate 🌐 Google 📊 Data Science Mind...

Jenkins / Terraform_job / #1 / Stages

Search

terraform apply -auto-approve 0.52s

```
7
8 # local_file.my_pet will be created
9 + resource "local_file" "my_pet" {
10   + content           = "i love pe3ts!"
11   + content_base64sha256 = (known after apply)
12   + content_base64sha512 = (known after apply)
13   + content_md5        = (known after apply)
14   + content_sha1        = (known after apply)
15   + content_sha256      = (known after apply)
16   + content_sha512      = (known after apply)
17   + directory_permission = "0777"
18   + file_permission     = "0777"
19   + filename            = "pets.txt"
20   + id                  = (known after apply)
21 }
22
23 Plan: 1 to add, 0 to change, 0 to destroy.
24 local_file.my_pet: Creating...
25 local_file.my_pet: Creation complete after 0s [id=01f2776342ec18f56195a04c5ea79bd146bc1d18]
26
27 Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

↑ ↓